ActInf ModelStream 010.1 ~ Building a Drone with RxInfer.jl ~ Bart van Erp, Albert Podusenko

Source: YouTube **Playlist**: ModelStreams **Duration**: 1:03:48 | **Uploaded**: Unknown **Transcript method**: auto_caption

---

hello and welcome this is active inference model stream number 10.1 on April 4th 2024 4424 and we're really excited today to be with Bart benp and Albert penko talking about building a drone with RX and fur we have an ongoing RX and fur learning group at The Institute and a lot of participants have been excited to use even early versions of this package and connect it to all the exciting math and developments so thank you both for joining looking forward to this very pragmatic presentation awesome thanks Daniel thanks for the kind's words and nice introduction also welcome everyone online um as we Daniel mentioned we're going to build a drone today so last week I didn't know anything about drones whatsoever but in the meantime I was able to craft one with the help of my colleagues and I want to kind of transfer the learning experience that I obtained over this week uh to all of you such that you can Leverage The Power of our toolbox RX AER in the end to make your own very cool applications your own drone your own autonomous vehicle whatever you want and we will then be there also to support you in this journey but for now let's get started how are we going to build a drone with RX infer so first of all let me Bri briefly introduce myself my name is Bon Arab I'm a BD student and teaching assistant in Bap which is the research group here at inov University of Technology and together with Albert who will be Al who's also live uh and we'll also be monitoring the chat in case there any questions pop up along the way we will be your host today together with Daniel um this talk will be supported by reactive base so reactive Bas is the overcoupling getup organization of RX infer and all the packages on which it's built at the moment we're looking for contributors so if you have if you're interested after this talk please reach out and then definitely we can uh get you acquainted with the toolbox and with our environment [Music] there so we're from bias laab the aov University of Technology we are located here in aov called city of light and from here we're doing very interesting research on building agents probalistic graphical models and using these to craft all kinds of engineering applications trying to solve real world problems with our engine our engine RX infer you

might have already heard about this but our engine um is is a Julia engine which allows us to perform automatic basan inference specifically through reactive message passing the engine that we have built is completely reative itive which means that it doesn't do anything unless there is something to do for it so it's relatively energy efficient because it doesn't consume energy uh during all the other moments and we perform message passing but I'll give a more in detailed discussion on this more explanation will follow later in the stock and we're going to use this toolbox in order to craft a drone a drone that can actually fly using some simplified drone mechanics go from position a to position B and I'm very eager to show you how this journey how this development Journey looks like so let's get started so what I'm going to talk about are three parts basically in this do I'm going to first start off with the model specification so in the basan inference methodology the first part is always that we need to craft a model and this model will include some drone physics so I'll the take you back to high school where you learned all these Concepts refresh a couple of them and based on that I'm going to construct a generative model of how a simplified drone would look like furthermore I'll then continue upon the probabilistic inference so if we have crafted this model and we want this model actually to do something inference allows us to compute posteriors in this model and the inference on this model is actually the underlying control algorithm that we will be crafting and this will be automated using our toolbox RX Ander then finally we'll wrap up with some experiments and then there will be plenty of time uh to answer any questions that you might have let's get going for the model specification we're going to start off with some simplified drones physics there will be very easy for some a refresher for others but hopefully throughout this uh throughout these following couple of minutes you'll have a basic understanding of how a drone works we have greatly simplified it because of course going into the details of three uh three-dimensional drones with wind all that kind of stuff would be a bit too much for just a single live stream but we hope to offer you the tools and maybe the Machinery to create these more complicated models yourself so we have this very nice drone here and what we're interested in is what forces act on this drone so we're going to simplify this a bit and we'll say that each of the rotors so the left and the right rotor produce some kind of upward Force pulling this drone from the from the ground of course we also walk around on Earth and we know that there is also a force that opposes this namely gravity which tries to pull us to the ground well combining these two forces allows us to either increase or decrease the altit altitude of this drone and make it Fly that's the entire goal in the end the goal will be to come up with these forces these FL and F FR such that our drone flies from location a to a

location B if we do the computations a bit and see what are the kind of the net force forces acting on this drone is in this case we see that we have a vertical component F1 which is the sum of the individual forces created by the motor minus the force of gravity that's acting upon the Drone and in the horizontal Direction because everything is nicely aligned we have a net force of zero if there would be wind for example there would be a contribution there but in the simplified model I assume that well there is no horizontal component as of yet but if we tilt the Drone a bit with a certain angle Theta we actually see that this Theta not only affects of course the horizontal um or the vertical net force but also the horizontal one so based on this we can obtain equations for this horizontal and vertical force acting upon the Drone the motors we assume that they are at a distance r a radius are from the center of mass so we very simplistically assumed that the center of mass of the Drone is at the radius R from its rotor so the force you can think of it as acting upon an arm creating a specific moment so the torque that's gets generated by these rotors with respect to the center of mass of the Drone is the difference between these two forces multiplied with with the radius r with these three equations so the net force is acting upon the Drone and this net torque that we can compute based on the rotation that the Drone wants to go to we can already construct a lot big part of the model of this drone so these are kind of the forces that we that act upon the Drone horizontal vertical Direction the rotation is given by this torque and we want to convert these forces into motion because in the end we want to move the Drone and we want to figure out what forces are required to move from a position a to a position B using Newton's law we can already know that our force is equal to our u a mass multiplied with our acceleration in other words our acceleration is our Force divided by our Mass So based on the net forces that we just computed we can also extract equations body accelerations in the vertical and horizontal Direction these accelerations accumulate so if we integrate over them we get actually a velocity so an incremental change in our velocity you can think of is actually this acceleration times this incremental change in time which gives us kind of a relationship so if we increase the acceleration over period of time then we also know that that will lead to an increase in our velocities in both the horizontal and vertical direction if we take this even a step further is then we can also start to model the actual position of the Drone so integrating over this velocity will give us our position in both the X and Y Direction basically every Small Change in X will be a result of the particular velocity over a small period of time and with this set this this set of equations it actually becomes possible to extract the movements from these net forces of the Drone so that's basically the movement so how it moves from left to right up and down but

there is this additional component to the Drone namely its angle because its angle also changes the acceleration in the angle so the angular acceleration Alpha here is the torque divided by the moment of inertia of the Drone you can think of this as being very similar to Newton's equation with the forces and the mass which give us the horizontal or vertical acceleration but this is just the similar uh similar equation that's used for actually angular acceleration similarly we can also compute the angular velocity so Omega because of small increment in this angular velocity will be our angular acceleration times a small fraction of time so integrating over the angular acceleration will give us the angular velocity and you might have already guessed this based on this we can also compute a new angle so the angle is this velocity integrated over time very similarly as we derived the equations for the actual movement of the Drone and together these net forces these equations for the movement of the Drone and these simplified equations for the rotation they constitute the Drone Dynamics so it might have seemed like a recap from high school but for the simplified example this is all that we actually need in order to craft this drone and that's what I'm going to show you right now because these equations of motion I'm going to write down in a function specifically a Julia function because that's the language that RX infer also uses on the hood so we're going to create this function f and I'm going to walk you right through it there are just 15 lines of code so I think this should be manageable so we have a function f which accepts the state so you can think of this state as the positions velocities angle and angular velocities of the Drone uh the actions to which we refer to the forces on the left and right rotor the particular drone like what are the characteristics of that drone and the environment so are we on Mars are we on Earth all those attributes that are um environment related so in the first three lines what we're going to do is we're going to extract some information so we're going to extract the forces from our actions we're going to extract the mass the moment of inertia and the radius of the Drone because these are properties that correspond to the particular drone that we're using and the gravity we're going to just grab from our environment because if we would be on Mars we would have very different dynamics of course then from the state what we extract are X and Y which are vertical or horizontal and vertical positions VX and VY which are velocities in the horizontal and vertical Direction Theta which was our angle and Omega which was our angle angular velocity with all these properties characteristics and States extracted we can actually start implementing the equations that we have on the left so FG which is the force generated by the gravity is simply equal to the mass times the gravitational constant FY and FX are merely copied from the equations on the left so it's the sum of the forces multiplied with coine or sign

of the angle and for the Y Force the net y Force we also subtract the gravity because there it has an influence and for the net torque we do the similar so we extract the difference between the force generated by the left and right rotor and multiply that with the radius in order to get us a torque then next up is we can based on these forces we can compute the accelerations in the X and Y Direction so F ofx / M will give us these accelerations or F of Y id M for the vertical acceleration we're going to take a very Cru Oiler approximation so we're going to do a linearization in order to actually create a new velocity so VX new which will be the velocity after enforcing the actions will be our previous velocity plus our acceleration times this increment in in time this D D and this DT we said beforehand but this is a simplified assumption which of course there are more complicated assumptions to be made we could choose uh oil Marana and all those other fourth order approximations but for now let's keep it simple and let's stick to this very simple order approximation then based on the velocities and accelerations we can also do a similar trick for the new positions so how does our X and Y coordinate change based on their previous coordinates the velocities that we had at the previous moment and the accelerations that we computed here we have this quadratic term over the accelerations uh which are merely a matter of fact of the approximation because we over DT we also want to kind of integrate over the accelerations so that's ODS um the second term basically ODS for this effect and makes it a bit more realistic with that out of the way we can do similar things for the rotation so the acceleration the rotational acceleration is our torque divided by our moment of inertia that we got from the Drone we again can update the angular velocity based on its previous angular velocity the acceleration that we just computed and the time interval and based on that we can also update our angle so what this function does it accepts the state actions and some characteristics of the Drone in the environment and some discretization over time and it computes and returns a new state so if we would have applied a specific action of force FL and F FR corresponding to the two rotor engines what would be the new state or approximately the new state that we end up with it's very simplified but this you can think of as a state transition function with it out of the way now it's actually time to start with the easy work because we have now this function f and this function f specifies how we think this internal state of the Drone evolves over time something I forgot to mention but Julia also supports Unicode so the people who are paying attention might have already seen it I still think it's a great feature but you can actually use Greek symbols which makes this translation going from physics to code way more easy and also allows you to retain this physical interpretation to it but that being said let's continue with the proper model so just remember

now that we have F which en codes our state transition for our drone we're going to construct a generative model and here I have drawn it as a so-called for St Factor grow but I'm going to talk you right through it so what we have is these boxes and these boxes correspond to factors or functions on how variables relate to each other which one relates to another one and how are they interconnected through which functions and through um through which nodes let's say and the edges so the arrows here in this case correspond to the individual variables in our model the variables s in this example correspond to the state of the agent so that contains the position of the agent in both x y and the velocities also in the XY Direction furthermore this state also contains the angle and this angular velocity to make life a bit simpler u in this case corresponds to the actions so the actions that we want in the end to also infer based on this figure we can for example see around this function f which was our state transition that we put in a previous state and the actions and we get something out to that we add a bit of gausian noise to accommodate for any uncertainties that are maybe in the parameters in the um in the environment because we use simplified drone physics maybe we want to add a bit of process noise there just to accommodate for the fact that these Dynamics are not perfect and that there will always be internal ex external fluctuations that drive this drone if we write this down in a probalistic matter we can construct this particular Vector graph where we have these time slices which repeat over time and mathematically we can write that as in the equation that's on the slide so we have a prior on the initial state which we refer to as P of s0 which encodes our starting position starting ities starting angles and all the way at the end of the model we have our goal prior so P of s where capital T promotes the length of our model also known as our Horizon this P of St describes okay where do we actually want the agent to go what do we want its velocity to be at the end what do we um want this angle to be at the end all these kind of things so together P of s0 and P of St these two priors the enode the starting and end position of our drone respectively whereas this middle part with the F function the transition function that we just described and the control U of which we can put a gion prior because we want to infer it together these encode the Dynamics of the Drone the generative model how does this this look like I'm just going to Port it directly into RX infer so over the past couple of months with our colleagues have spent um significant amounts of time actually improving our modeling language which makes it now even easier to craft your own probalistic modelss in RX infer and with these effectively four lines five lines of code we can construct the model the graph that we have on the left so what I'm going to do is I'm going to talk you through it and I think at the end I think hope all questions or I I don't think there will be a lot of questions

surrounding this particular model any longer because it's merely a replication of what's on the left we're going to Define this variable S1 and S1 in this case actually refers to the first state but judia indexes with one so that's why it's not S zero but S1 in instead and S1 which is our encodes our starting position we're going to encode that with a multivariate normal distribution which main which mean is given by some initial state that we provide to the function and which covariance Matrix we assume to be very narrow because we assume that we know the position of the drone at the initial time step then inside this for loop we're going to basically unroll this middle segment of the Drone of this generative model we're going to create a prior on the control so on this U oft and we're going to put there Al multivariant normal because that's nice to work with w mean is the mass times gravitation constant divided by two and then as a vector of two the reason for this is because our prior Assumption of this drone is that it exerts a force because we think it's it's flying so it would make sense that we assume the force of the engines will be just enough maybe to balance the Drone in the air so this mg ided by two which is actually the force of the left and right rotor respectively they compensate for the force of gravity if the mo if the engine or the Drone were to be upright so here we provide some additional information that allow us to imp improve convergence and improve also estimation because we know that the Drone is probably not going to drop out of air we want it actually to kind of stabilize and stay at its position so we do that here by putting this prior on control then following that is our state transition so how do we think our state evolves over time and this is where the Dynamics of the simplified run that we talked about on the previous slide actually come into play so the mean of the next state will be given by this function f over the previous state and the corresponding actions and the Drone the environment and the discretization over time together with some noise because of the unknown Dynamics maybe we've got some internal external fluctuations uncertainties stochastic Behavior so that's all something that we also encode in this multivariate normal distribution so that incorporates this for Loop and that this for Loop basically unrolls this segment that's on the left containing this P of U this F and this Norm then finally we finish up our model by putting this goal prior at the end so at the Horizon plus one because of the indexing we're going to put the multivariate normal whose mean is located at the goal containing our goal position goal the velocities that we wish to attain at the goal um the angle at goal etc etc and together these couple lines of code effectively they're just five they create the generative model that's on left so that's great we have crafted a model of how we think the Drone evolves over time we have derived some simplified equations to model it states

Dynamics and now we want to do inference so we are interested in Computing the marginal distributions over both actions and States basically we want to ask our model okay it's nice that you have encoded this starting and end position in the model but in order to obtain that how can we actually reach that how can we actually do that so we're interested in figuring out what is actually the marginal distribution over the actions so this Q of U and what is the marginal distribution over the states so what forces do we need to apply to go from position a to position B and how will this trajectory going from A to B look like in the general case if we were to just try to solve this equation and compute these marginal distributions then we would have to integrate over all other parameters but that's very computationally heavy because there are a lot of parameters in a model so doing this in a naive way would not be efficient at all but there is luckily a way to make it efficient namely through message passing so let's dive a bit deeper into how that actually looks like so for this purpose I've created this simplified factorized model to make sure everyone can kind of follow along so we have the different factors fa FB FC and FD each link to some of the variables from X1 ranging to X6 and and each Vector is only connected to the variables to which it also relates it's a simple case right suppose now that we wish to compute the marginal here over the variable X4 doing this naively would require us to integrate out this factorized model over all variables X except for X4 however that's computation yav as I mentioned earlier because we have variables to integrate over and that might be a challenge because we're in this very high dimensional space then luckily what we can do is we can make use of that these factors are not connected to everything so the factor nodes that are on the left are not connected to each of the variables and this turns out to be very beneficial from an efficienty standpoint so we can effectively split up this large integration over the entire model FX into a set of smaller integrals simply by making use of this distributive property of integration where we can kind of separate them out put some parts into brackets and by doing so this large integration problem actually reduces into a couple smaller integration problems which are more efficient to solve we can also give an interpretation to this for for example this fa of X1 X2 integrated over the variable X1 you can also think of this as closing the book as we demonstrate on the left over that factor F fa and this is the result of this we call a message so this message you can think of as a summary of a particular part of that particular graph of part of graph and we can iterate this a couple of times so this message mu of X2 if we propagate that into the next node this FB and we solve the corresponding integration with this double integration over now X2 and X3 we get a new message mu of X4 and we can do this this on all the edges and all the over all the factor nodes

here in our model we can integrate over the single node FD over the Lo FC where we have an incoming message mu of X5 and what we end up with and that's the interesting part is that all these messages are relatively easy to compute they are relatively small because they do not require this huge integration the funny thing here is is that this marginal the one that we're interested in this Q of X4 is now simply product of mu of X4 * the other mute so the messages that are actually colliding on that edge and then of course renormalized but that marginal distribution can simply be computed on a very local level by multiplying the messages that are flowing over these edges hence making it very efficient to solve this integration problem and this is also how we actually solve inference in our toolbox so in the probalistic generative model that we created for the we can use message passing to solve this marginal marginalization in a very efficient manner to give you an example these are some benchmarking results that we obtained with RX infer on the left you can see how long it takes for a simple stage based model which is similar to the model that I've shown before uh but you can think more of this like in a common filter setting for people who have a bit familiarity with Control Systems uh and we can see that the filtering and the smoothing both scale very well they scale very nicely in the number of observation or in the length of the model on just linearly basically on the right we also do a comparison between RX and fur so R toolbox and churing churing is another toolbox in judia which doesn't use message passing to solve the actual uh marginalization problem but instead it reverts to sampling so instead of computing everything kind of analytically or mostly analytically or with approximations it draws a lot of samples and tries to extract statistics from this but as you can see the left axis is on the loog scale so we obtain significant performance improvements in comparison to the sampling based methods and that's also why we are pushing for our iner because it's just way more efficient to perform these competitions with message passing in comparison to sampling based methods there is one thing in our model so if you recall these drone Dynamics then our state transition was quite nonlinear so it was very challenging or it is very challenging to do exact message passing with this nonlinear note because if we go from the previous state to the next state then the distribution of this next state will no longer be nice and gaan but it might be uh distribution which is outside of the EXP the distribution or the outside of the family of exponential distributions let's say so it requires some approximations in order to actually do inference around it so in the experiments I have used unscented which we can just specify with a little code block on the right and what this does it draws a very small but very informative set of samples around the incoming cauan distributions which capture the first and second moment

of the distribution so the mean and variances of these Sigma points these samples these are still the same as the statistics of the incoming normal distribution what we then do is we use these very carefully select samples to propagate throughly nonlinearity and based on the transformed Sigma points we then reconstruct what the gion at the output could have looked like so this unscented allows us to make approximations uh through this nonlinearity with that in place we can actually perform inference so the code block on the left bottom is all you need in order to get the Drone running so inside the infar function we specify a model which was the Drone model that we specified in one of the previous slides we specify its data although it's not observing something directly and it's more doing a planning task we can still Supply it with the initial State and the goal state where do we want the agent to move to and this meta object corresponds to the approximations that we wish to take and that's all that's that's all that's necessary in order to actually get this drone up and running and in order to demonstrate this we have a very nice experiment created by Dimitri one of our colleagues where we have this drone moving around it will be it's more complicated than or I would say the plotting is more complicated than what I've shown on the previous slide because actually getting this plotting to work requires way more code than actually getting our INF to work uh but what it allows us to do is not only to do this planning but also to do this really in an online manner so here in this particular example we have created this uh this video where on the Fly we are changing parameters so we're increasing the mass of the Drone making it heavier we're increasing the size of the Drone U so it's radius it's engine power like what are the limits of this uh this drone what can it L gravity perhaps you want to move to another planet and test your simulation there but we're changing these characteristics on the Fly and with RX you're still able to process kind of this do this planning task that being said um all the code and the example that I showed you today they are publicly available on the reactive base organization so in the RX for examples. repository you can find the code um code required to run actually the Drone experiment it will be a simplified experiment so not the one on the previous slide because that would be way too intense to over that would be just too overwhelming to have a look at but the main characteristics of this particular drone demo are available here and if you have any questions want to try it out please let me know after um or contact me if you have issues um but the code that we developed for this particular live stream is actually publicly available and with that I would like to thank you of course for this very nice opportunity for this nice audience um if you want to have more information feel free to reach out to any one of us reactive base file issue whatever you've seen uh deem pretty can contact us research group

bias laab our spin-off lace Dynamics for industrial applications but most importantly for reactive base I want to say that we also looking for collaborators so if you got motivated by this star got inspired to create your own cool applications please reach out to us and hopefully help us develop our ex infer into the toolbox of the future think that's something that I would like to strive for so if you're in inspired reach out and we'll make sure that there are amazing opportunities lying ahead and that's all thanks a lot for attention and I hope it was an inspiring talk that got you engaged with ARX and fur got you excited so uh if you have any questions please let me know awesome thank you very cool presentation Albert do you want to give any first or overview remarks and then Chris and kopus we can discuss and ask some questions too uh well not not really in a sense that yeah it was although it was the first time I saw this presentation it was uh great B I mean I I knew the the underline project behind that and how the Drone works but the presentation I think was very nice to also connect how uh while the bean inference is basically how is it connected to the gold driven Behavior I think it's um it's really cool um yeah I just wanted to add along the presentation that Bart was speaking about this noise component when he drew when he has drawn the graph so this noise component also could be seen as basically the the compensation for our poor approximation of the Dynamics right because we use this first order linear approximation and well that's arguably not not the best uh way of approximating the underlying physics although it worked for the simplified demo and there well I think I can just maybe add on how this demo can be extended or this model can be extended um well the first thing to do is well just to use a different approximation right uh that that's uh one thing that comes to my mind um and another thing would be and I think from the active inference perspective uh would be to Omit some of the parameters in the transition function deem them unknown and to let the agent figure out it uh through the interaction with the environment so you could achieve that also with um AR infer by putting some priors on the on the metrix or putting some prior on the components of this metrix and you can um see how fast the agent actually learns the physics of the environment yeah awesome I'll ask some quick questions from the live chat and then Chris and kobus if you want to ask feel free okay so Carlos asks is the reactive part based on events to trigger the pipeline or based on signals yeah yeah go ahead Bart yeah so was we or what Dimitri did when he build it when he started building this RX infer toolbox uh um is he used a very different Computing Paradigm so normally you execute code from the top to the bottom but what he did is he used this reactive Paradigm So based on events a particular computation is being executed and these events can be various it can change of course it depends very much

in the context what an event is but often the event corresponds to making actual observations so especially for active inference agents if we make a new observation only then recomputations are being executed so basically the entire algorithm does nothing unless there is something to compute and I hope that kind of answers the question but I'm not entirely sure so if I was a bit unclear maybe Albert you can uh out to it if you have a better yeah I'm just not I'm not sure which kind of reactive part was uh was meant in in the question because yeah there is the reactive part of the inference but also the demo itself was reactive right like when you are triggering this but but basically speaking about how how it works indeed it it it the it Triggers on the event basically so and the event is that yeah something there are few things that could be changed right it's either environmental change something changed in the environment uh and then we we sense this uh from from our sensors and then the inference follows right so there is uh just to add upon what BART said about this U reactive part so message passing is wellknown algorithm I mean it's it's it's been around uh since Pearl belief propagation perhaps later earlier but then uh the reactive is is a particular implementation of this message passing which is uh more more kind of uh driven by how the the the uh the system should work in the field uh yeah awesome yeah with Magnus kudal we explored a lot of the message passing talked a little bit about how all the messages could be passed at once or the reactive Paradigm where you could uncouple the rates of updating um okay I'll ask one more question from the live chat and then Chris oropus Fraser asks fantastic stuff double exclamation point uhoh if somebody wanted to help develop RX and fur with your organization what are the ideal characteristics and backgrounds to have I think we are very open so also within our lab uh in BIOS we have a huge variety of backgrounds range from computer science to physics electrical engineering for myself for example so there is a wide variety of backgrounds um possible of course we assume that or we would like coding to be something that you're good at or willing to learn um because I think that's one of the most important things of course to develop a software engine and all the rest I think we can provide you with excellent materials to get crossed or to uh to get familiar with the methods that we use or get acquainted with the models that we uh that we have developed in the past so personally when I started in in this group and when I started working on this project I also didn't have any background so I used to be an electrical engineer I never heard about well I was thought probability Theory and then quickly forgot about it um so my background was very different and it is maybe a bit difficult to get started directly with all this probably message passing and all this entire Paradigm but it's definitely a very nice ride to learn so I

would say um if you're willing to learn willing to make a nice contribution then that's always welcome yeah yeah I well we are mostly writing stuff in in Julia right so Julia is uh sort of I mean I mean it's learnable language right it's not like if you don't know Julia you can't uh help us no that's not absolutely not true so we we welcome uh people from different programming backgrounds uh and besides there are different branches on how you can help within this ARX in ferent basically the whole reactive base ecosystem because it cons consists not only of the inference engine which I must say it's the most difficult part for contributing because there are many intricacies uh involved that uh with within this uh inference language but um there is also graph PPL right there is also rocket there are other U packages which uh which basically constitute the whole ecosystem and uh yeah well you you don't have to be an expert in basian inference to contribute to Rocket uh or you don't have to be again um an expert in particular approximations to contribute to model builder so and of course uh documentation is something which is uh which always needs uh uh an update right so we we don't dismiss people for contributing to the documentation that's extremely important uh for the for the community and for ourselves because we we can see okay so uh where are the parts which we could explain better or actually improve our ecosystem awesome Copus ah very interesting talk thank you very much um I have a question about uh categorical examples where the the control space consists of values uh from a categorical distribution um so does orx infer have any such examples available we do we do so we do not only support continuous variables in this case but we also support a variety of category um Renity distributions uh categorical distributions themselves uh we do have a lot of examples so if if you visit the website RX in.ml there is this huge list of examples use cases uh some of which feature categorical distributions as well modeling is yeah all the ranging from modeling and simple coin tools to more advanced examples but we do offer functionality in both the discrete domain as in the continuous domain yeah okay thank yeah go ahead go ahead now I just wanted to add there is well most uh kind of famous example on on or hello world examples into the categorical world within I think is is hidden Market model and I think we have an example on that I'm not sure that we have uh Incorporated the control within that uh example that's probably learning the transition matrices for observations and the and the well the hidden State um but uh there is I think nothing stopping you from introducing the control for a for a certain uh discrete type of uh Mark model uh thank you and then um I use vs code um but I do have problems with Greek symbols uh when they contain multiple characters in a superscript for subscript for example so do you guys use V code at all and if so do you have a best practice uh to to solve my problem

we do use vs code I think our entire lab uses FS code because it has turned out to be a very very nice tool so it's good that you mention this um regarding the subscripts and the superscripts that you mention is personally I try to avoid them because often times I completely agree that it looks nice to have this dot over a variable for example to the note der ative or to draw kind of the index with a superscript but personally to me that clutters it a bit and also makes it more challenging to edit so I just try to refrain from using sub and super scripts in vs codes with this Auto tab although it looks very cool and might be nice for like demo purposes for actually experimentation or development I always try to avoid them so not a solution but uh this is my kind of my take on it okay yeah I can understand uhh yeah because and another possible problem with super and subscripts is they can appear very small and I often find myself having to zoom in a bit to you know discriminate between certain symbols and then my final question uh to what extent can you guys parallelize within RX in fur are there op is low hanging fruit choices to to apply parallelization in RX infer yes so um it's good that you mention this because at the moment we have actually a master student who is kind of working on this particular or trying to work on this project uh where we try to parallelize the message passing and the computation for example of the marginals so there are opportunities there to do some sort of parallelization um in general at the current moment our message posting is all linear unfortunately however there have been papers where they also paralized that although it seems that it's impossible to do apparently it still is uh but that's something perhaps for a future project so at the moment we do not provide a lot of parallelization around RX and fur but in the future this is definitely on top of our agenda to make also improvements in terms of performance and in computational speed please um just one quick question please um another one um have you guys thought approaching uh big players like Amazon web services for example to uh you know and present this to them and try and get them to incorporate it into their S tools we have been um so with L Dynamics kind of the spin-off company which also uses RX Ander to develop real application let say for customers we have considered this this option um but at the moment we have not been in contact to any of these these companies or clients um sorry no I just wanted that to we we haven't talked to none of uh Fang uh so that's that's uh yeah we haven't done yet although we are we are we in talks there are a few interested parties in the pipeline but uh no we have approached Amazon no not yet excellent thank you Kus I'll ask another question in the chat and then Chris if you have anything Fraser asks what are the largest current challenges to the reactive message passing Paradigm for doing this kind of approximate basian inference it's a very good question I think

it also depends a lot on who you ask this question to to me personally um the message passing I think is a great method in itself and it also overcomes already a lot of challenges what I think would be the most fruitful next step is to really start working on the actual modeling the modeling aspect so can we create maybe Universal models uh hierarchical models how can we do this such that these hierarchical models or similar actually solve more complicated tasks than a simple stage-based model can do um so for me personally I think that's where we can achieve the biggest gains in this modeling part I'm not sure about what Alber things are the most big what this yeah what the biggest outstanding challenge is but there are a I I think from from yeah I totally agree from the from the research perspective this structural adaptation of of the model that's the most I would say yeah the hardest and the most interesting uh from from my opinion problem that's like handcrafting this model like for for this type of drones right it's nice can be easy sometimes not well if you go to the three-dimensional that gets significantly uh more complex so indeed like having um a Machinery well the thing the interesting thing that we we do have Machinery to to do structural adaptation uh because aren fur uh does provide the an opportunity to extend your model to patch it uh although how to do that in principle how do you indeed grow or or shrink your model and so in time while observing and being being in the field I think this is the most challenging part indeed and as for well there are certainly uh problems with uh approximations and um uh well uh Universal rules uh that's something we also I think uh outline in a few demos that well there could be a situation where the rule for computing the message which B showing is not is not available uh so there is always this question right whether you want to um well to commit to a simpler approximation to do it FAS or you want to commit to a more accurate oximation but it will take more resources uh computational resources and how to balance between these two that's that's another interesting uh question how to how to automatically decide um I think I think this is what I would call the most interesting uh yeah yeah challenges I would also add from our kind of educational and research side that one of the big challenges thankfully one being approached is to devel velop the documentation and the examples the examples help us learn but also a lot of us are having good success bringing in code examples into code and language models and using the working examples to template new examples so as the library of open-source published models grows that will become more possible and then also there's a lifelong fascinating journey of talking about how do you go from seeing a drone in the sky to getting those physics equations down and that kind of like approach to modeling and how do we get to a graph assuming that that's where the software package can pick up and render it kind of

no problem but there's still this very human very collaborative process which also could receive a lot of documentation and examples about getting to that kind of a model Chris yeah absolutely Yeah Yeah Chris do you w to ask anything no I just wanted to say thank you guys very much I really enjoyed the presentation um I liked I really enjoyed how you broke down a lot of the top of my head right now is like the integration example um it it was very intuitive it made a lot of sense and it really highlighted a lot of the benefits and true power in uh what you guys have developed so thank you guys and I really really appreciate uh your time and explain this to us thank you for the kind's words if I could also add thanks for having us one um comment just as we've been working on it week by week a a real like arising Insight that got us both excited and not daunted but just realizing the scope of the the package and the work was in comparing the RX confer active inference examples with other active inference simulations like from pmdp or built kind of custom outside of a package like the INF for ants examples and we realized that a lot of alternative approaches people try to go as fast as possible to developing an active inference agent and then talk about like the variational free energy or the expected free energy of the agent and so the package serves as a helpful accelerator to get to an agents but that's not like the general agents and then even making small changes to that scheme can be buried and Scattered across different packages and then there's all these other kinds of limitations that one quickly finds thems encountering like in the education application settings whereas in RX and fur it's such an engineering driven approach so a lot of the engineers on our team were immediately more comfortable and then a lot of people who had been playing with active inference from a computer science or from a Phil philosophical and mathematical side were a little bit surprised because we were getting very low level with the node engineering and then after wiring that up and understanding it it's just like press play so a lot of the focus on developing an agent was actually that Focus was moved to better understanding the graph that represents the ecosystem of shared intelligence and there was less upfront focus on merely making it like an agent-based model because you could make an agent-based model like the Drone or you could develop other kinds of graphs that don't necessarily have an agentic basis so that helped us realize that this is a much broader toolkit than doing active inference or agent-based modeling of any kind but that those are use cases that are benefited greatly by it definitely I think you are actually piing the main point of our package so RX allows us to enable or enables us to create these agents very quickly very adapt very efficiently so if we compare this for example to and you mentioned this already also from your engineers like if you would go through a traditional design cycle for an

engineering firm you would create a single model kind of thingy single algorithm let's say 50 pages or of code or something and that's nice these 50 pages of code but all yeah it's unmaintainable you need a lot of Engineers to actually make improvements upon it yeah if something changes you know don't know what's going to happen right whereas with rifer we just have to specify this little piece of code which um we which specifies the model and then inferences automated as you mentioned you just press play it isn't that easy of course from the actual engine perspective but we try to make the user experience as good as possible to make it as easy on the user to build agents active inference agents or any other applications where you might need such Machinery to actually complete a specific task and hopefully by doing so by making this user friendliness one of our priorities is we can also enable active inference getting adopted across Industries and across research groups yeah um after today if we have a quick question is there an email that we could send those questions to I think there from the active inference Institute there is an Communication channel but Daniel please correct me if I'm wrong directly to bias laab but if you have questions feel free to start a discussion on GitHub because we actively maintain it we keep also um a close watch on what's Happening and there might also have been people with similar questions before you so my first hunch would be create a discussion on GitHub and we will definitely then get involved with one of our colleagues in order to help you out okay thanks well if if if you have questions uh regarding uh this Dynamics uh Venture then uh you could uh yeah use our info ATL dynamics.com as well so just to dump any other questions you want yep Co well thank you all again it's really awesome I was thinking about how the packages are kind of factorized and modular and separable almost like that's a theme that helps ecosystems work so overall thank you very much for the work and for presenting we've really appreciated the chance to work with a package in our learning groups and we'll look forward to seeing more examples and sharing more also from the projects that people are working on in The Institute thank you much was a pleasure thank you for coming us okay great so see you all next time bye bye bye byee thank youbody for